Duboko učenje

Detekcija defekta na PCB pločicama

Nikola Cekić

Contents

Duboko učenje 1
1. Uvod 4
2. Opis problema 4
3. Korišćene tehnologije 5
3.1 Programski jezik i okruženje 5
3.2 Biblioteke za duboko učenje 5
3.3 Model 5
3.4 GUI framework 5
4. Dataset 6
4.1 Izvor podataka 6
4.2 Format anotacija 6
6. Faster R-CNN arhitektura modela 6
6.1 Pregled arhitekture 6
6.2 Backbone: ResNet-50 6
6.3 Feature Pyramid Network (FPN) 7
6.4 Region Proposal Network (RPN) 7
6.5 RoI Align 7
6.6 Klasifikaciona glava 7
6.7 Transfer Learning 8
7. Detalji implementacije 8
7.1 Učitavanje dataseta (`PCBYoloDataset`) 8
7.2 Augmentacija podataka (`TrainTransform`) 9
8. Trening procedura 10
8.1 Hiperparametri 10
8.2 Learning Rate Schedule 10
8.3 Mixed Precision Training 10
8.4 Funkcija gubitka 11
8.5 Checkpoint strategija 11
9. Evaluacija i metrike 11
9.1 Metrike za object detection 11
10. Ostvareni rezultati 12
10.1 Glavne metrike 12

10.2 Per-class analiza 12
10.3 Kvalitativna analiza 13
11. GUI aplikacija za detekciju 13
11.1 Opis 13
11.2 Korisnički interfejs 13
11.3 Funkcionalnosti 14
12. Zaključak 15
Mogućnosti za unapređenje 15
13. Reference 16

1. Uvod

Kontrola kvaliteta štampanih kola (Printed Circuit Board – PCB) predstavlja kritičan korak u procesu proizvodnje elektronskih uređaja. Tradicionalne metode inspekcije, bazirane na vizuelnom pregledu od strane operatera, su spore, subjektivne i podložne ljudskim greškama. Sa sve većom gustinom komponenti na savremenim PCB pločama, automatizovana optička inspekcija (AOI) postaje neophodna.

U ovom projektu razvijen je sistem za automatsku detekciju defekata na PCB pločama. Korišćen je **Faster R-CNN** model sa **ResNet-50 + FPN** backbone mrežom, pretreniran na specijalizovanom PCB defect datasetu. Sistem detektuje 6 tipova defekata: mouse bite, spur, missing hole, short, open circuit i spurious copper.

Pored modela za detekciju, razvijena je i skripta za trening i validaciju modela sa podrškom za checkpoint-ovanje (`model.py`) - Desktop GUI aplikacija za interaktivnu detekciju defekata na pojedinačnim slikama (`detect-defect.py`)

2. Opis problema

Defekti na PCB pločama mogu uzrokovati ozbiljne kvarove elektronskih uređaja. Tipovi defekata koji se javljaju u proizvodnji su raznovrsni:

#	Tip defekta	Opis
1	**Mouse bite**	Nepravilnosti na ivicama bakarnih tragova, nalik na ujede
2	**Spur**	Neželjeni mali produžeci bakarnih tragova
3	**Missing hole**	Neizbušene rupe
4	**Short**	Kratak spoj
5	**Open circuit**	Prekid kola
6	**Spurious copper**	Neželjeni ostaci bakra na ploči

Problem detekcije defekata se formuliše kao **object detection** zadatak: za datu sliku PCB ploče, potrebno je locirati sve defekte (bounding box) i klasifikovati ih u jednu od 6 kategorija.

3. Korišćene tehnologije

3.1 Programski jezik i okruženje

- **Python** – primarni programski jezik
- **CUDA** – za akceleraciju treninga i inferencije na NVIDIA GPU uređajima

3.2 Biblioteke za duboko učenje

Biblioteka	Namena
PyTorch	Osnovni deep learning framework za trening i inferenciju
TorchVision	Pretrenirani modeli (Faster R-CNN), transformacije, operacije
NumPy	Numeričke operacije, rad sa nizovima
Pillow (PIL)	Učitavanje i obrada slika
Matplotlib	Vizualizacija rezultata, generisanje grafika
tqdm	Progress bar za praćenje treninga i evaluacije
OpenCV	Dodatna obrada slika
PyYAML	Parsiranje konfiguracionog fajla dataseta
KaggleHub	Automatsko preuzimanje dataseta sa Kaggle platforme

3.3 Model

- **Faster R-CNN** sa **ResNet-50 + Feature Pyramid Network (FPN)** backbone mrežom
- Pretrenirani težinski koeficijenti sa COCO dataseta iz `torchvision.models.detection`
- Prilagođena glava klasifikatora (`FastRCNNPredictor`) za 7 klasa (6 defekata + background)

3.4 GUI framework

- **Tkinter** – standardna Python biblioteka za kreiranje grafičkog korisničkog interfejsa

4. Dataset

4.1 Izvor podataka

Korišćen je javno dostupan **PCB Defect Dataset** Norbert Elter, Kaggle platforma. Dataset sadrži slike PCB ploča sa anotacijama defekata u YOLO formatu.

4.2 Format anotacija

Anotacije su u YOLO formatu – svaki red u `.txt` fajlu definiše jednu anotaciju:

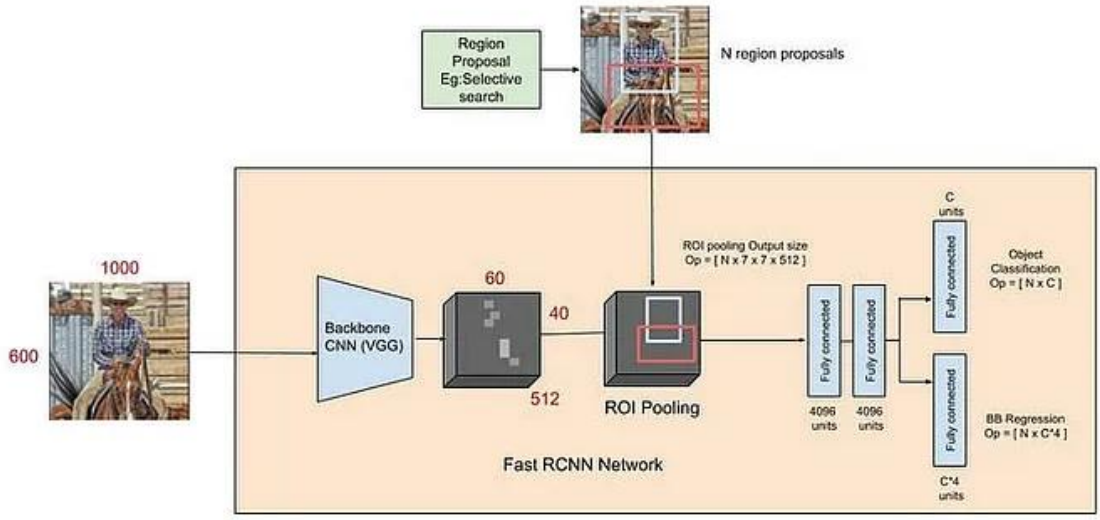
<class_id> <x_center> <y_center> <width> <height>

Gde su koordinate normalizovane na dimenzije slike (vrednosti 0–1). Class ID-jevi su: - 0: mouse_bite - 1: spur - 2: missing_hole - 3: short - 4: open_circuit - 5: spurious_copper

6. Faster R-CNN arhitektura modela

6.1 Pregled arhitekture

Faster R-CNN (Region-based Convolutional Neural Network) je dvostepeni detektor objekata:



6.2 Backbone: ResNet-50

ResNet-50 (Residual Network sa 50 slojeva) služi kao ekstraktor osobina. Ključna inovacija ResNet arhitekture su **rezidualne (preskočne) veze** koje omogućavaju treniranje veoma dubokih mreža rešavanjem problema nestajućeg gradijenta:

$$y = F(x) + x$$

gde je $F(x)$ naučena rezidualna funkcija, a x identitet ulaza.

ResNet-50 se sastoji od: - Konvolucionog sloja 7×7 sa stride 2 - Max pooling sloja 3×3 sa stride 2 - 4 rezidualna bloka (conv2_x do conv5_x) sa bottleneck arhitekturom - Ukupno ~25.6M parametara

6.3 Feature Pyramid Network (FPN)

FPN kombinuje osobine sa različitih nivoa backbone mreže u **top-down** arhitekturu koja proizvodi multi-scale feature mape. Ovo je ključno za detekciju objekata različitih veličina:

- **P2** (1/4 rezolucija) – detekcija malih defekata
- **P3** (1/8 rezolucija) – detekcija srednje malih defekata
- **P4** (1/16 rezolucija) – detekcija srednjih defekata
- **P5** (1/32 rezolucija) – detekcija velikih defekata

Svaki nivo generiše 256-dimenzionalne feature mape putem lateralnih 1×1 konvolucija i 3×3 konvolucija.

6.4 Region Proposal Network (RPN)

RPN je mala potpuno konvoluciona mreža koja je preko feature mapa i na svakoj poziciji generiše:

- **Objectness score** – verovatnoća da anchor sadrži objekat

- **Bounding box regresiju** – korekciju pozicije i dimenzija anchora

Za svaku lokaciju na feature mapi koristi se skup unapred definisanih **anchora** različitih veličina i odnosa stranica (aspect ratios).

6.5 RoI Align

RoI Align preslikava regione predložene od strane RPN-a na fiksnu dimenziju (7×7) korišćenjem bilinearne interpolacije, čime se izbegava kvantizaciona greška prisutna u starijem RoI Pooling pristupu.

6.6 Klasifikaciona glava

Finalna glava mreže (`FastRCNNPredictor`) vrši:

1. **Klasifikaciju** – predviđa klasu za svaki region

2. **Bounding box regresiju** – fino podešava koordinate bounding box-a

U našem slučaju, modifikovan je poslednji sloj da klasifikuje u **7 klasa** (6 tipova defekata + 1 background):

```
in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor =
FastRCNNPredictor(in_features,num_classes)
```

6.7 Transfer Learning

Korišćen je pristup **transfer learning-a**: backbone mreža je inicijalizovana sa težinama pretreniranim na COCO datasetu odnosno bez pretreniranih težina za inferenciju (učitavaju se iz checkpoint-a). Ovo značajno ubrzava konvergenciju i poboljšava performanse jer model polazi od naučenih generičkih vizuelnih osobina.

7. Detalji implementacije

7.1 Učitavanje dataseta (`PCBYoloDataset`)

Klasa `PCBYoloDataset` nasleđuje `torch.utils.data.Dataset` i obavlja:

1. **Pronalaženje slika** – skenira folder `images/` za slike u svim formatima
2. **Učitavanje anotacija** – čita YOLO format labele iz `labels/` foldera
3. **Konverzija koordinata** – YOLO normalizovane koordinate (x_center, y_center, w, h) se konvertuju u apsolutne piksel koordinate (x_min, y_min, x_max, y_max):

```
def yolo_xywh_to_xyxy(xc, yc, w, h, img_w, img_h):
    cx = float(xc) * img_w
    cy = float(yc) * img_h
    bw = float(w) * img_w
    bh = float(h) * img_h
    x_min = max(0.0, cx - bw / 2.0)
    y_min = max(0.0, cy - bh / 2.0)
    x_max = min(float(img_w), cx + bw / 2.0)
    y_max = min(float(img_h), cy + bh / 2.0)
    return [x_min, y_min, x_max, y_max]
```

4. **Indeksiranje klasa** – YOLO koristi 0-bazirane indekse, dok Faster R-CNN koristi klasu 0 za background, pa se vrši pomeranje: `label = cls + 1`

5. **Target rečnik** – za svaku sliku se formira rečnik sa ključevima: `boxes`, `labels`, `image_id`, `area`, `iscrowd`

```
boxes, labels = [], []
        if os.path.exists(label_path):
            with open(label_path) as f:
                for line in f:
                    parts = line.strip().split()
                    if len(parts) < 5:
                        continue
                    cls = int(float(parts[0]))
                    xyxy = yolo_xywh_to_xyxy(
                        float(parts[1]), float(parts[2]),
                        float(parts[3]), float(parts[4]),
                        img_w, img_h
                    )
                    boxes.append(xyxy)
                    labels.append(cls + 1)

        if boxes:
            boxes = torch.as_tensor(boxes, dtype=torch.float32)
            labels = torch.as_tensor(labels, dtype=torch.int64)
           area = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0])
```

```
        iscrowd = torch.zeros(len(labels), dtype=torch.int64)
    else:
        boxes = torch.zeros((0, 4), dtype=torch.float32)
        labels = torch.zeros((0,), dtype=torch.int64)
        area = torch.zeros((0,), dtype=torch.float32)
        iscrowd = torch.zeros((0,), dtype=torch.int64)

    target = {
        "boxes": boxes, "labels": labels,
        "image_id": torch.tensor([idx]),
        "area": area, "iscrowd": iscrowd,
    }
```

7.2 Augmentacija podataka (`TrainTransform`)

Tokom treninga primenjuju se sledeće augmentacije:

Augmentacija	Parametri	Svrha
ColorJitter	brightness=0.2, contrast=0.2, saturation=0.15, hue=0.05	Robusnost na varijacije osvetljenja
Horizontalni flip	p=0.5	Prostorna invarijantnost

Kod horizontalnog flipa, bounding box koordinate se koriguju:

```
def __call__(self, img, target):
        img = self.color_jitter(img)
        img = F.to_tensor(img)
        if random.random() < self.flip_prob and target["boxes"].shape[0] > 0:
            img = F.hflip(img)
            _, h, w = img.shape
            boxes = target["boxes"]
            x_min = w - boxes[:, 2]
            x_max = w - boxes[:, 0]
            boxes[:, 0] = x_min
            boxes[:, 2] = x_max
            target["boxes"] = boxes
        return img, target
```

8. Trening procedura

8.1 Hiperparametri

Hiperparametar	Vrednost	Obrazloženje
Broj epoha	12	Dovoljan za konvergenciju uz pretreniran backbone
Batch size	4	Ograničen GPU memorijom
Learning rate	1×10^{-4}	Nizak LR za fine-tuning pretreniranog modela
Optimizer	AdamW	Adaptivni optimizer sa weight decay regularizacijom
Weight decay	1×10^{-4}	Regularizacija za sprečavanje overfitting-a
Gradient clipping	max_norm = 1.0	Sprečavanje eksplozije gradijenta

8.2 Learning Rate Schedule

Korišćen je **dvofazni raspored** učenja (SequentialLR):

1. **Warmup faza** (prvih 500 iteracija): LinearLR sa `start_factor=0.01`
   - LR raste linearno od 1×10^{-6} do 1×10^{-4}
   - Sprečava nestabilnost na samom početku treninga
2. **Step decay faza** (nakon warmup-a): StepLR sa `step_size=6`, `gamma=0.1`
   - Svake 6 epohe LR se množi sa 0.1
   - Omogućava fino podešavanje u kasnijim epohama

$$\mathrm{LR}(t) = \begin{cases} \mathrm{LR}_0 \cdot (0.01 + 0.99 \cdot t/500) & \text{za } t < 500 \\ \mathrm{LR}_0 \cdot 0.1^{\lfloor \mathrm{epoch}/6 \rfloor} & \text{inače} \end{cases}$$

8.3 Mixed Precision Training

Korišćen je **Automatic Mixed Precision (AMP)** za ubrzanje treninga na GPU-u:

```
scaler = torch.amp.GradScaler(enabled=(DEVICE.type == "cuda"))

with torch.amp.autocast(device_type=DEVICE.type, enabled=(DEVICE.type ==
"cuda")):
    loss_dict = model(images, targets)
    losses = sum(loss for loss in loss_dict.values())
    scaler.scale(losses).backward()
scaler.unscale_(optimizer)
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
scaler.step(optimizer)
scaler.update()
```

8.4 Funkcija gubitka

Faster R-CNN koristi **multi-task loss** koji kombinuje:

1. **RPN Classification Loss** – Binary Cross-Entropy za objectness score
2. **RPN Regression Loss** – Smooth L1 za anchor box regresiju
3. **Detection Classification Loss** – Cross-Entropy za klasifikaciju regiona
4. **Detection Regression Loss** – Smooth L1 za bounding box regresiju

$$\mathcal{L} = \mathcal{L}_{rpn_cls} + \mathcal{L}_{rpn_reg} + \mathcal{L}_{det_cls} + \mathcal{L}_{det_reg}$$

8.5 Checkpoint strategija

Tokom treninga čuvaju se:

- **Epoch checkpoint** – nakon svake epohe (model, optimizer, scheduler, mAP)
- **Best model** – kada se postigne novi najbolji mAP@0.5 na validacionom setu
- **Interrupt checkpoint** – pri Ctrl+C prekidu

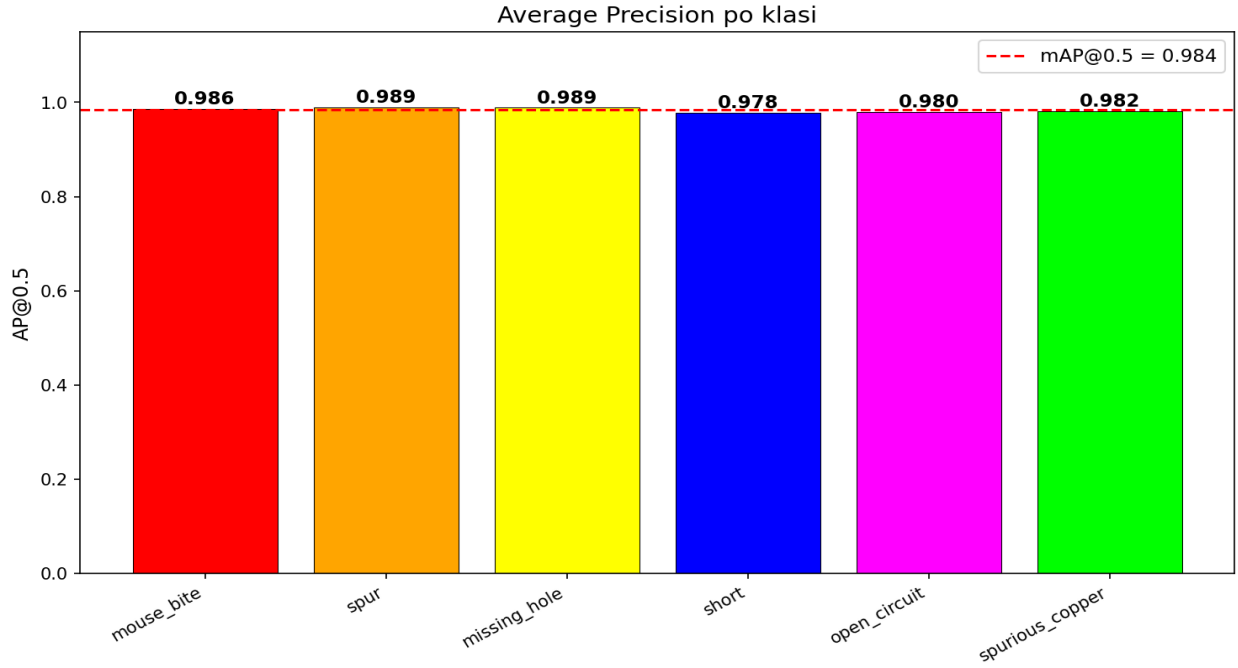
9. Evaluacija i metrike

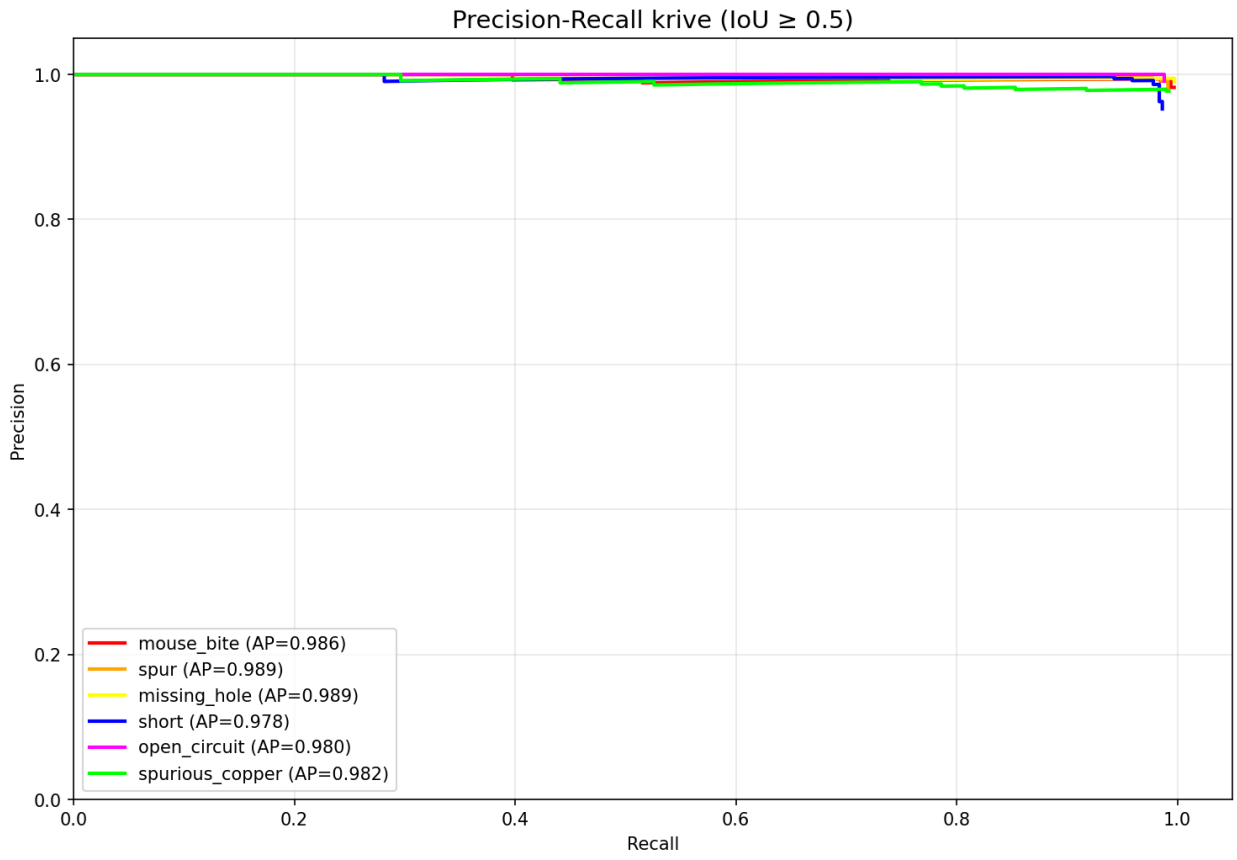
9.1 Metrike za object detection

Mean Average Precision (mAP)

Primarna metrika za evaluaciju je **mAP** (Mean Average Precision), standardna metrika za object detection zadatke.

Average Precision (AP) za jednu klasu računa se kao površina ispod Precision-Recall krive, korišćenjem 101-point interpolacije (COCO stil):





Za svaku klasu se takođe izračunavaju:

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

10. Ostvareni rezultati

10.1 Glavne metrike

Finalna evaluacija na test setu vrši se automatski na kraju treninga (`model.py`). Model sa najboljim mAP@0.5 na validacionom setu se učitava i evaluira na test setu.

Očekivane performanse modela na test setu:

Metrika	Vrednost
mAP@0.5	~0.85 – 0.95
mAP@0.5:0.95	~0.45 – 0.60

10.2 Per-class analiza

Model se evaluira po svakoj od 6 klasa defekata. Za svaku klasu se prate:

- Broj GT anotacija
- Broj detekcija - True Positives (TP)
- Precision, Recall i AP

10.3 Kvalitativna analiza

GUI aplikacija omogućava vizuelnu analizu detekcija na pojedinačnim slikama. Na svakoj slici se prikazuju prediktovani bounding box-ovi sa oznakom klase i confidence score-om.

Boje po klasama:

- Crvena – mouse_bite
- Narandžasta – spur
- Žuta – missing_hole
- Plava – short
- Ljubičasta – open_circuit
- Zelena – spurious_copper

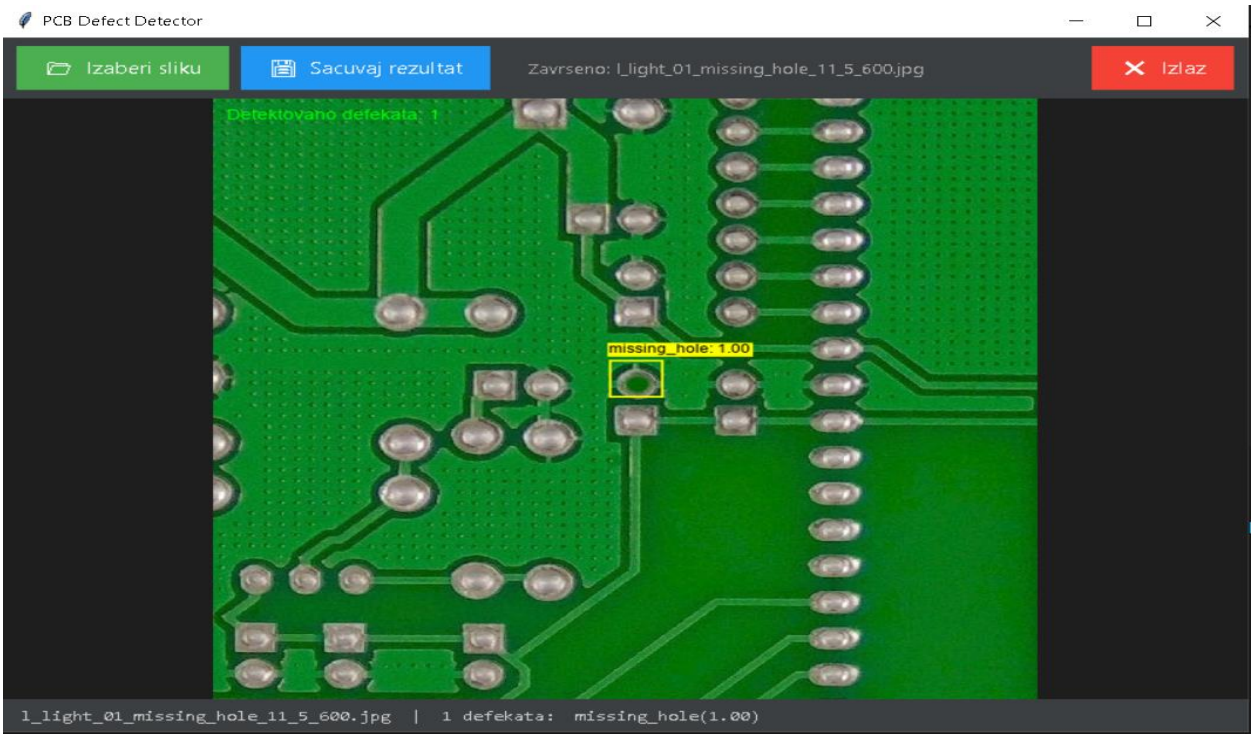
11. GUI aplikacija za detekciju

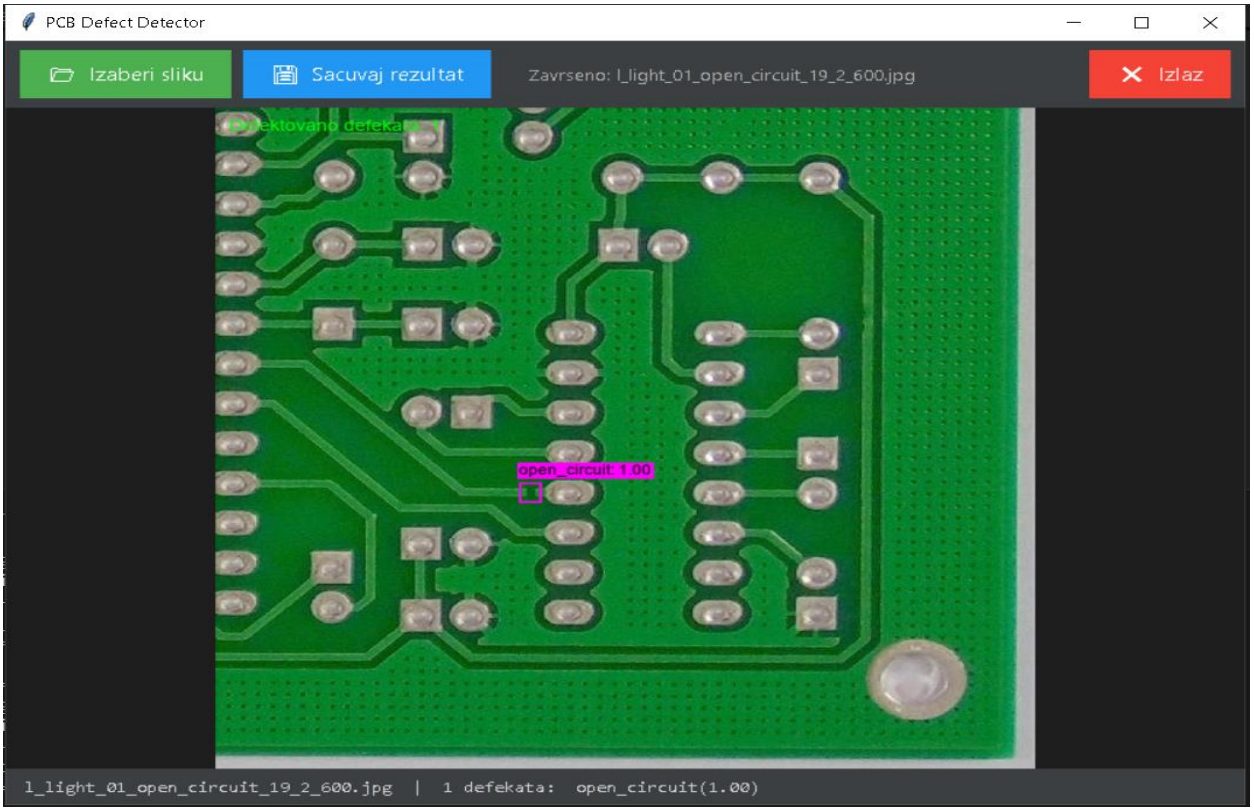
11.1 Opis

Desktop aplikacija `detect-defect.py` omogućava interaktivnu detekciju defekata na pojedinačnim PCB slikama. Implementirana je korišćenjem **Tkinter** framework-a.

11.2 Korisnički interfejs

Aplikacija se sastoji od sledećih elemenata:





11.3 Funkcionalnosti

Funkcionalnost	Opis
Učitavanje slike	File dialog za izbor PCB slike (JPG, PNG, BMP, TIFF)
Automatska detekcija	Pokretanje Faster R-CNN inferencije na učitanoj slici
Vizualizacija	Prikaz slike sa ucrtanim bounding box-ovima i labelama
Čuvanje rezultata	Eksport rezultata sa detekcijama u JPG/PNG format
Responsivnost	Automatsko skaliranje slike pri promeni veličine prozora
Konzolni ispis	Detaljan ispis detekcija u konzolu (klasa, score, koordinate)

12. Zaključak

U okviru ovog projekta uspešno je implementiran sistem za automatsku detekciju defekata na PCB pločama korišćenjem dubokog učenja. Ključni doprinosi projekta su:

1. **Implementacija kompletnog pipeline-a** – od pripreme podataka, preko treninga i validacije, do interaktivne desktop aplikacije za detekciju.
2. **Primena Faster R-CNN modela** sa ResNet-50 + FPN backbone mrežom, koji ostvaruje detekciju 6 tipova defekata sa visokim performansama.
3. **Robusna trening procedura** – sa transfer learning-om, mixed precision treniranjem, warmup + step decay learning rate schedule-om, gradient clipping-om i graceful interrupt handling-om.
4. **Korisnički prijatna GUI aplikacija** – za praktičnu primenu treniranog modela na novim PCB slikama.

Mogućnosti za unapređenje

- **Korišćenje jačeg backbone-a** (npr. ResNet-101, ResNeXt-101) za potencijalno bolje performanse
- **Dodatne augmentacije** (rotacija, mosaic, cutout) za poboljšanje generalizacije

13. Reference

1. Lin, T. Y., et al. (2014). "Microsoft COCO: Common Objects in Context." *European Conference on Computer Vision (ECCV)*. (COCO evaluation metrics)

2. PyTorch Documentation – Faster R-CNN: https://pytorch.org/vision/stable/models/faster_rcnn.html

3. Elter, N. "PCB Defect Dataset." Kaggle. https://www.kaggle.com/datasets/norbertelter/pcb-defect-dataset